DESENVOLVIMENTO MOBILE

AULA 7 — NAVEGAÇÃO AVANÇADA COM EXPO ROUTER E PUSH NOTIFICATIONS

Rotas dinâmicas · Tabs · Drawer · Notificações · CESAR School 2026.2

AGENDA

1. **Rotas dinâmicas**: `[id]`, parâmetros e `useLocalSearchParams`
2. ✋ Checkpoint 1
3. **Tabs**: layout com múltiplas abas
4. ✋ Checkpoint 2 · ☕ Intervalo
5. **Drawer**: menu lateral e animação
6. ✋ Checkpoint 3
7. **Push Notifications**: permissão, token, listeners
8. ✋ Checkpoint 4
9. Hands-on guiado · Atividade aplicada

AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

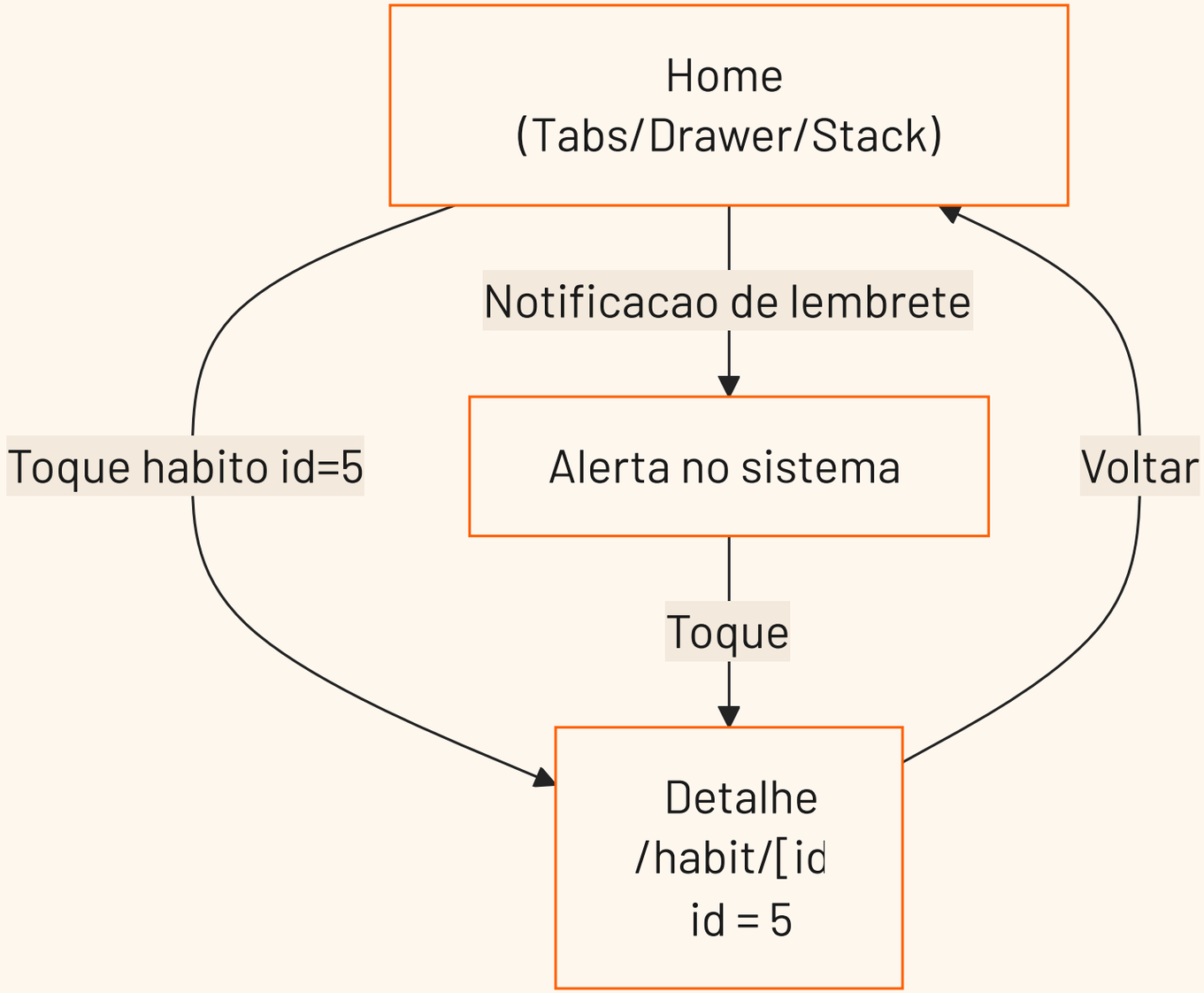
1. Implementar **rotas dinâmicas** com `[id].tsx` e navegar com `href` em objeto
2. Ler parâmetros com `useLocalSearchParams()`
3. Montar um layout de **abas** com `<Tabs>`, configurando ícones
4. Montar um **drawer** com `<Drawer>` e abrir por gesto ou por código
5. Decidir entre **Stack, Tabs e Drawer** para um caso concreto
6. Descrever o fluxo de **notificações**: permissão → token → listeners
7. Integrar notificações locais, reconhecendo as limitações do simulador

O QUE ENTRA HOJE

- **Rotas dinâmicas** — `[id].tsx`, `[...rest].tsx`, `useLocalSearchParams`
- **Tabs** — `<Tabs>`, `Tabs.Screen`, ícones
- **Drawer** — `<Drawer>`, gestos, dependências de animação
- **Push Notifications** — permissão, token, listeners, agendamento local

> Um navegador por vez. Compor Drawer + Tabs + Stack e push **remoto** ficam para depois.

PARÂMETROS, NAVEGADORES E AVISOS



O APP COMEÇA A LEMBRAR DO USUÁRIO.

PARTE 1

ROTAS DINÂMICAS

A GAVETA DE FOTOS

Uma rota **dinâmica** é uma tela cujo conteúdo depende de um **parâmetro** — em vez de `/sobre`, você tem `/habit/5` e `/habit/42` abrindo a **mesma tela** com dados diferentes.

- 🗂️ **Rota estática:** sempre a mesma coisa — `/sobre`, `/configuracoes`
- 🗃️ **Rota dinâmica:** detalhe de item, edição, busca — `/habit/[id]`

SINTAXE: COLCHETES SÃO ESPECIAIS

Arquivo	Aceita qual URL	Parâmetros
`habit/[id].tsx`	`/habit/123`	`{ id: "123" }`
`habit/[id]/edit.tsx`	`/habit/123/edit`	`{ id: "123" }`
`[...rest].tsx`	`/qualquer/coisa`	`{ rest: [...] }`

Convenção: minúsculas com hífen — `[habit-id].tsx`, não `[HabitId].tsx`.

NAVEGAR PARA ROTA DINÂMICA

```tsx
// Link com objeto
<Link href={{ pathname: '/habit/[id]', params: { id: '5' } }}>
  Ver hábito 5
</Link>

// router.push com objeto
router.push({
  pathname: '/habit/[id]',
  params: { id: '5' }
})
```

O parâmetro entra via `params`, sempre string.

LER OS PARÂMETROS

```tsx
import { useLocalSearchParams } from 'expo-router';

export default function HabitDetail() {
  const { id } = useLocalSearchParams<{ id: string }>();

  useEffect(() => {
    fetchHabit(id);
  }, [id]);

  return <Text>Hábito: {id}</Text>;
}
```

`useLocalSearchParams()` é o padrão. `useGlobalSearchParams()` é raro.

REMOUNT VS. ATUALIZAÇÃO

```
router.push({ pathname: '/habit/[id]', params: { id: '5' } });
router.push({ pathname: '/habit/[id]', params: { id: '6' } });
// A segunda chamada REMONTA o componente
```

Consequência: a lista no fundo é destruída. Voltar com `back()` a traz de volta.

✋ CHECKPOINT 1 — ROTAS DINÂMICAS

1. Quando usar rota dinâmica vs. estática?
2. Arquivo `pet/[id]/edit.tsx` — qual a URL? E os parâmetros?
3. Abri `/pet/5` e depois `/pet/6`. Remontou ou só atualizou?
4. Como navegar para `/pet/5` passando `name: 'Rex'`?

⏱️ 60 segundos em duplas

PARTE 2

TABS

ABAS LADO A LADO, NÃO PILHA

Tabs é um navegador onde várias telas convivem num **menu em abas** — como no Instagram: Home, Explorar, Perfil. Diferente de Stack, Tabs **não empilha**.

- Cada aba é uma tela independente, com seu próprio scroll e estado
- Trocar de aba **preserva** o estado — como as abas do navegador

TABS — ESTRUTURA DE PASTAS

```
src/app/
  (tabs)/
    _layout.tsx        ← Define as abas
    index.tsx          ← Aba 1
    search.tsx         ← Aba 2
    profile.tsx        ← Aba 3
```

> A pasta `(tabs)` com parênteses **não vira rota** — é transparente.

TABS — IMPLEMENTAÇÃO

```tsx
// app/(tabs)/_layout.tsx
import { Tabs } from 'expo-router';

export default function TabLayout() {
  return (
    <Tabs screenOptions={{ tabBarActiveTintColor: '#007AFF' }}>
      <Tabs.Screen
        name="index"
        options={{
          title: 'Home',
          tabBarIcon: ({ color }) => <Icon name="home" color={color} />
        }}
      />
      <Tabs.Screen name="settings" options={{ title: 'Settings' }} />
    </Tabs>
  );
}
```

STACK × TABS × DRAWER

	Stack	Tabs	Drawer
Navegação	push/back	Toque aba	Desliza lateral
Estrutura	Pilha	Menu embaixo	Menu do lado
Quando	Sequencial	Seções paralelas	Acesso global

✋ CHECKPOINT 2 — TABS

1. Por que a pasta se chama `(tabs)` e não `tabs`?
2. Onde você define os ícones das abas?
3. Desce a lista em Home, toca Search, volta — a lista continua onde?

⏱️ 60 segundos em duplas · ☕ Intervalo a seguir

PARTE 3

DRAWER

A GAVETA LATERAL

Drawer é um **menu lateral** que desliza de fora da tela. Diferente de Stack e Tabs, o Drawer é **modal** — quando aberto, flutua por cima.

- iOS: arrastar da borda esquerda
- Android: deslizar de lado ou botão no cabeçalho

DRAWER — INSTALAÇÃO OBRIGATÓRIA

```
npx expo install react-native-reanimated \
  react-native-worklets react-native-gesture-handler
```

Sem essas 3 libs, o Drawer **não funciona**. Instalar antes da aula.

DRAWER — IMPLEMENTAÇÃO

```tsx
// app/_layout.tsx
import { Drawer } from 'expo-router/drawer';

export default function DrawerLayout() {
  return (
    <Drawer screenOptions={{ drawerActiveTintColor: '#007AFF' }}>
      <Drawer.Screen name="(tabs)" options={{ drawerLabel: 'Home' }} />
      <Drawer.Screen name="about" options={{ drawerLabel: 'About' }} />
      <Drawer.Screen name="settings" options={{ drawerLabel: 'Settings' }} />
    </Drawer>
  );
}
```

ABRIR O DRAWER POR CÓDIGO

```
import { useNavigation } from '@react-navigation/native';

const navigation = useNavigation();

<Pressable onPress={() => navigation.openDrawer()}>
  <Text>≡ Menu</Text>
</Pressable>
```

Import de `@react-navigation/native`, não de `expo-router`.

✋ CHECKPOINT 3 — DRAWER

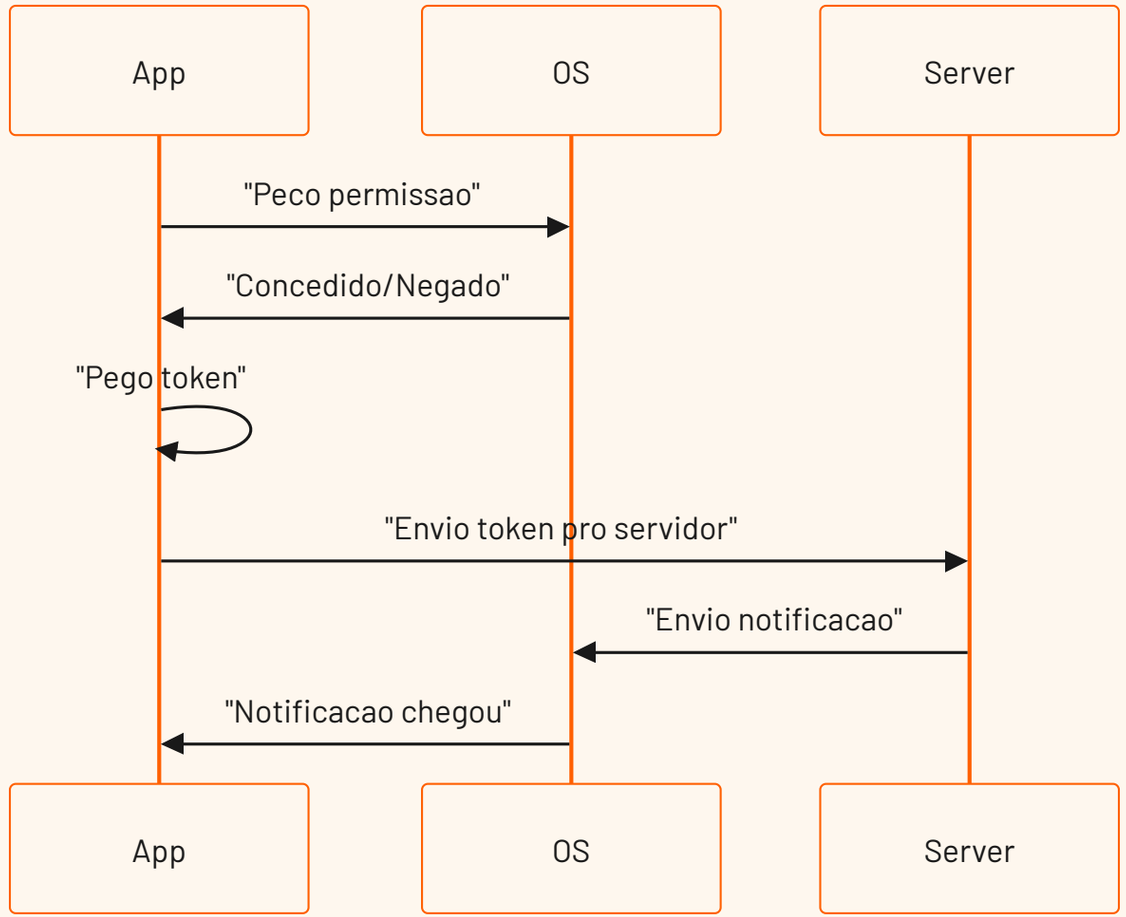
1. Três libs instaladas — quais são?
2. Pasta `(tabs)` dentro de Drawer — o que significa?
3. Como abrir o Drawer sem gesto (por código)?

⏱️ 45 segundos em duplas

PARTE 4

PUSH NOTIFICATIONS

NOTIFICAÇÕES — O FLUXO COMPLETO



Hoje: até "Notificação chegou". Push remoto é depois.

PERMISSÃO

```
import * as Notifications from 'expo-notifications';

const { status } = await Notifications.requestPermissionsAsync();
if (status !== 'granted') {
  console.log('Permissão negada');
  return;
}
```

iOS: dialog "O app gostaria de enviar notificações. Permitir?"

PEGAR O TOKEN

```
import Constants from 'expo-constants';

const projectId = Constants?.expoConfig?.extra?.eas?.projectId;
const token = await Notifications.getExpoPushTokenAsync({ projectId });
console.log('Token:', token.data);
// Envie para o backend aqui
```

Token identifica o dispositivo. Único por instalação.

COMPORTAMENTO EM FOREGROUND

```
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldPlaySound: true,
    shouldSetBadge: true,
    shouldShowBanner: true,
  }),
});
```

> **Sem isso:** notificação chega mas não mostra com app aberto.

LISTENERS E AGENDAMENTO LOCAL

```
Notifications.addNotificationResponseReceivedListener((response) => {
  // Usuário tocou — navegar para a tela certa
  router.push('/habit/' + response.notification.request.content.data.habitId);
});

await Notifications.scheduleNotificationAsync({
  content: { title: 'Hora do hábito!', data: { habitId: '5' } },
  trigger: { seconds: 5 },
});
```

✋ CHECKPOINT 4 — NOTIFICAÇÕES

1. Diferença entre `requestPermissionsAsync` e `getExpoPushTokenAsync`?
2. Quando `setNotificationHandler` é necessário?
3. No simulador iOS você testa push remoto?

⏱️ 45 segundos em duplas

TUDO JUNTO

```tsx
// Drawer (menu) + Tabs (seções) + Rota dinâmica + Notificação

// app/_layout.tsx → Drawer
<Drawer><Drawer.Screen name="(tabs)" /></Drawer>

// app/(tabs)/_layout.tsx → Tabs
<Tabs><Tabs.Screen name="index" /></Tabs>

// app/(tabs)/habit/[id].tsx → Rota dinâmica
const { id } = useLocalSearchParams<{ id: string }>();
// Notificação toca aqui ↓
// router.push({ pathname: '/habit/[id]', params: { id: '5' } })
```

PRÓXIMOS PASSOS (NÃO É AULA)

- Compor múltiplos navegadores (Drawer + Tabs + Stack)
- Push notificações **remotas** (Firebase, Expo Push Service)
- Deep linking (notificação abre a tela certa com URL)
- Persistência de estado (useContext, Redux)

SEU APP AGORA LEMBRA DO USUÁRIO

ROTAS DINÂMICAS · TABS · DRAWER · NOTIFICAÇÕES

Aula 7 — Desenvolvimento Mobile · CESAR School 2026.2